Instituto Politécnico do Cávado e do Ave
Escola Superior de Tecnologia
Licenciatura em Engenharia de Sistemas Informáticos

Integração de Sistemas de Informação

Trabalho Prático I

Processos ETL para Gestão de Stocks e Vendas em Comércio Eletrónico

Gonçalo Braga, nº 31508 (Pentaho Data Integration)
Gonçalo Rodrigues, nº 31507 (KNIME Analytics Platform)

Docente: Óscar Ribeiro
Barcelos, outubro de 2026

Índice

1. Enquadramento 3
2. Problema 4
2.1 Dados de entrada 4
3. Estratégia utilizada 5
3.1 Regras de negócio comuns 5
3.2 Operadores utilizados 5
4. Transformações 7
4.1 Pentaho Data Integration 7
4.2 KNIME Analytics Platform 8
4.3 Resultados obtidos 12
5. Jobs 14
5.1 Pentaho: job_etl.kjb 14
5.2 KNIME: orquestração com metanodes 15
6. Vídeo de demonstração 16
7. Conclusão e trabalhos futuros 17
8. Referências 18

1. Enquadramento

Campo	Descrição
Instituição	IPCA, Escola Superior de Tecnologia
Curso	Licenciatura em Engenharia de Sistemas Informáticos
Unidade Curricular	Integração de Sistemas de Informação (ISI), 2026/27
Docente	Óscar Ribeiro
Equipa	Gonçalo Braga (31508) e Gonçalo Rodrigues (31507)
Plataformas	Pentaho Data Integration (Spoon) e KNIME Analytics Platform 5
Repositório Git	`https://github.com/braguex10/ISI-TP1-Ecommerce-ETL`

Este relatório documenta o Trabalho Prático I da unidade curricular de Integração de Sistemas de Informação, cujo objetivo é aplicar e experimentar ferramentas de ETL (Extract, Transform and Load) na integração de dados provenientes de sistemas heterogéneos.

O trabalho foi realizado em grupo de dois elementos. Tal como exige o enunciado, cada elemento explorou uma plataforma diferente: o **Pentaho Data Integration** (Gonçalo Braga) e o **KNIME Analytics Platform** (Gonçalo Rodrigues). Ambos implementaram o mesmo processo ETL, sobre os mesmos dados de entrada, o que permitiu comparar as duas ferramentas e validar os resultados cruzando os valores obtidos.

2. Problema

Uma loja online de material informático recebe diariamente dados de três sistemas distintos, cada um com o seu formato e com problemas de qualidade:

- o **sistema de vendas** da loja exporta as encomendas em JSON, com os dados dos clientes e as linhas de cada venda aninhadas;
- o **sistema de armazém** exporta o catálogo de produtos e o stock em CSV;
- os **fornecedores** enviam as suas tabelas de preços de custo e prazos de entrega em XML.

A estes junta-se uma fonte externa: uma **API REST pública** (Frankfurter) que fornece as taxas de câmbio do dia, usadas para apresentar as vendas também em dólares e libras.

O objetivo é construir um processo ETL que integre estas fontes, limpe e valide os dados, proteja os dados pessoais dos clientes de acordo com o RGPD, calcule indicadores de negócio (totais, margens, resumo por categoria), separe as vendas válidas das rejeitadas, gere alertas de stock e disponibilize os resultados numa base de dados e em ficheiros de vários formatos.

2.1 Dados de entrada

Ficheiro	Formato	Conteúdo
`vendas_online.json`	JSON	3 transações com cliente (nome, email, NIF, telefone), método de pagamento e lista de linhas de venda (SKU, quantidade, preço)
`produtos_stock.csv`	CSV (;)	8 produtos com nome, categoria, preço, stock, armazém e data da última entrada
`fornecedores_precos.xml`	XML	2 fornecedores, cada um com os produtos que fornece (SKU, preço de custo e prazo de entrega em atributos)
API Frankfurter	JSON (REST)	Taxas de câmbio EUR→USD e EUR→GBP do dia da execução

Tabela 1: Fontes de dados

Os dados contêm, propositadamente, vários problemas de qualidade que o processo ETL tem de resolver:

Problema	Exemplo
Valor em falta	SKU-1003 sem valor de stock
Erro de escrita	Categoria "Monitroes" em vez de "Monitores"
Email inválido	`maria_santos_email_invalido` (sem @)
NIF em formatos diferentes	`PT234567890` e `210987654`
Telefones em formatos diferentes	`+351 912 345 678` e `961234567`
Produto sem fornecedor	SKU-1006 vendido mas ausente do XML de fornecedores
Venda sem stock	SKU-1005 vendido com stock 0

Tabela 2: Problemas de qualidade nos dados de entrada

3. Estratégia utilizada

O processo foi estruturado nas três fases obrigatórias do enunciado. Para garantir que as duas implementações eram comparáveis, o grupo definiu antes de começar as mesmas regras de negócio, os mesmos nomes de colunas e as mesmas tabelas de destino.

Fase	Objetivo	Operações principais
E Extração	Ler as 4 fontes heterogéneas	Leitura de CSV, JSON e XML; chamada à API REST; desdobramento de estruturas aninhadas
T Transformação	Aplicar a lógica de negócio	Limpeza de nulos, correção de valores, remoção de duplicados, validação e normalização com regex, anonimização, joins/lookups, cálculos, conversão de moeda, filtragem, agregação e ordenação
L Carregamento	Disponibilizar os resultados	Base de dados SQLite; exportação para CSV, JSON e XML; log de execução; orquestração (job / metanodes) e notificação por email

Tabela 3: Estrutura do processo ETL

3.1 Regras de negócio comuns

- **Email válido:** tem de respeitar a expressão regular de email (Tabela 6).
- **Venda válida:** o cliente tem email válido e o produto tem fornecedor conhecido (para ser possível calcular a margem). As restantes vão para a lista de rejeitadas.
- **Cálculos por linha:** `total_linha = quantidade × preco_aplicado`, `custo_total = quantidade × preco_custo`, `margem = total_linha − custo_total`, e conversão para USD e GBP com a taxa do dia.
- **Estado do stock:** RUTURA quando o stock é 0, CRITICO quando é inferior a 5, e OK nos restantes casos.
- **RGPD:** nenhum dado pessoal real (nome, email, NIF, telefone) é gravado nos destinos; apenas versões anonimizadas.

3.2 Operadores utilizados

Operação	Pentaho (step)	KNIME (nó)
Ler CSV	CSV file input	CSV Reader
Ler e desdobrar JSON	JSON input	JSON Reader, JSON Path, Ungroup
Ler XML	Get data from XML	XML Reader, XPath, Ungroup
Chamar API REST	Generate rows + REST client	GET Request + JSON Path
Valores em falta	If field value is null	Missing Value
Corrigir valores	Value mapper	String Manipulation (regexReplace)
Validar com regex	Regex evaluation	Rule Engine (MATCHES)
Normalizar e anonimizar	Replace in string (regex)	String Manipulation (regexReplace)
Remover duplicados	Unique rows (HashSet)	Duplicate Row Filter
Juntar dados	Stream lookup, Join rows	Joiner (inner e left outer), Cross Joiner
Cálculos	Calculator	Math Formula
Separar válidas/rejeitadas	Filter rows	Rule-based Row Splitter

Operação	Pentaho (step)	KNIME (nó)
Classificar stock	Number range	Rule Engine
Agregar e ordenar	Group by, Sort rows	GroupBy, Sorter, Column Renamer
Base de dados	Table output (SQLite)	SQLite Connector + DB Writer
Exportar ficheiros	JSON output, Text file output, XML output	CSV Writer, Table to JSON, JSON Writer, JSON to XML, XML Writer
Log	Group by, Join rows, Get system info, Text file output	GroupBy, Cross Joiner, Date&Time Range Creator, CSV Writer
Orquestração	Job (.kjb) com email	Workflow com metanodes

Tabela 4: Correspondência de operadores entre as duas plataformas

4. Transformações

4.1 Pentaho Data Integration

No Pentaho, todo o processo de extração, transformação e carregamento está numa única transformação, `pentaho/01_ler_dados.ktr`, com 46 steps. Os caminhos dos ficheiros são relativos (`${Internal.Entry.Current.Directory}`), pelo que a transformação funciona em qualquer computador sem alterações.

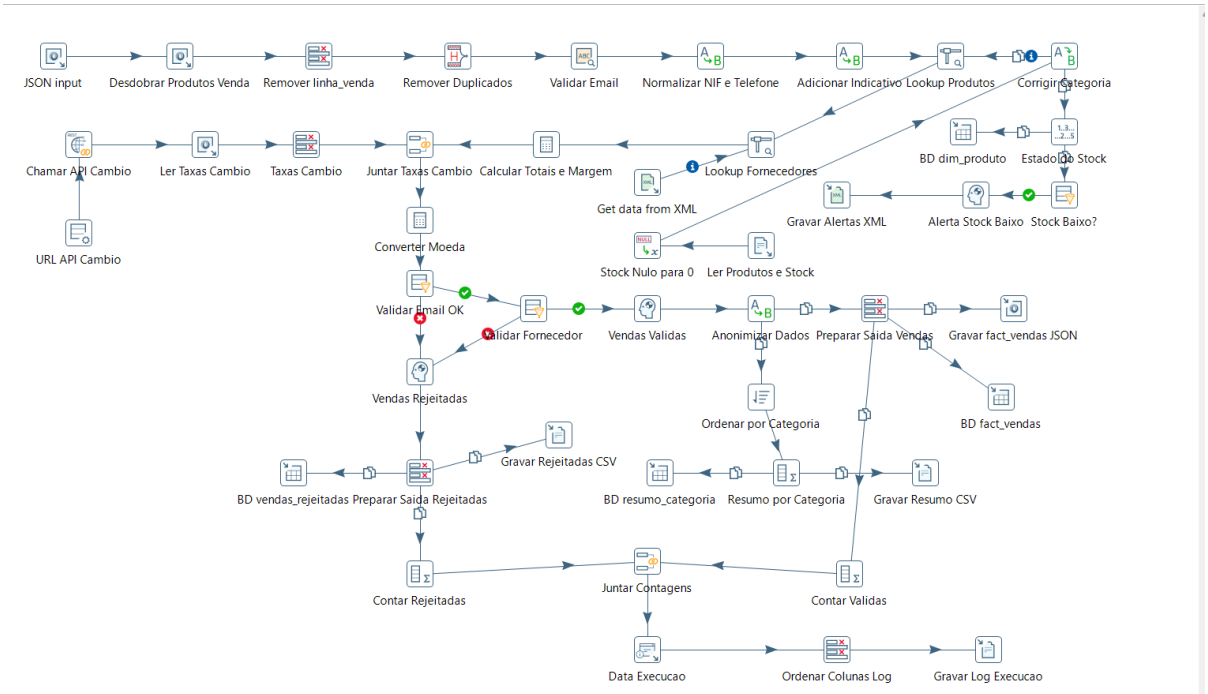


Figura 1: Transformação Pentaho 01_ler_dados.ktr

Extração

- **Ler Produtos e Stock** (CSV file input) lê o catálogo com separador `;`.
- **JSON input** lê as transações e **Desdobrar Produtos Venda** transforma o array `linha_venda` em várias linhas (uma por produto vendido).
- **Get data from XML** lê os produtos de cada fornecedor através de XPath.
- **URL API Cambio** (Generate rows) define o endereço da API, **Chamar API Cambio** (REST client) faz o pedido GET e **Ler Taxas Cambio** extrai as taxas USD e GBP da resposta JSON.

Transformação

- **Stock Nulo para 0** e **Corrigir Categoria** (Value mapper) tratam o stock em falta e a gralha "Monitroes".
- **Remover Duplicados** elimina linhas de venda repetidas.
- **Validar Email** (Regex evaluation) cria o campo `email_valido`.
- **Normalizar NIF e Telefone** e **Adicionar Indicativo** (Replace in string) uniformizam os formatos com regex.
- **Lookup Produtos** e **Lookup Fornecedores** (Stream lookup) acrescentam o nome, a categoria, o preço de custo e o prazo de entrega.

- **Calcular Totais e Margem** e **Converter Moeda** (Calculator), depois de **Juntar Taxas Cambio** (Join rows).
- **Validar Email OK** e **Validar Fornecedor** (Filter rows) separam as vendas válidas das rejeitadas.
- **Anonimizar Dados** (Replace in string) mascara o NIF, o telefone e o email.
- **Estado do Stock** (Number range) classifica cada produto; **Resumo por Categoria** (Group by) agrega as vendas válidas.

Carregamento

- **BD fact_vendas**, **BD vendas_rejeitadas**, **BD resumo_categoria** e **BD dim_produto** (Table output) gravam na base de dados `output/pentaho/ecommerce_pentaho.db`.
- **Gravar fact_vendas JSON**, **Gravar Rejeitadas CSV**, **Gravar Resumo CSV** e **Gravar Alertas XML** exportam os ficheiros.
- **Contar Validas**, **Contar Rejeitadas**, **Juntar Contagens**, **Data Execucao** e **Gravar Log Execucao** acrescentam uma linha ao log em cada execução.

4.2 KNIME Analytics Platform

No KNIME, o processo está no workflow `knime/TP1_Ecommerce_ETL.knwf`. Os nós estão organizados em blocos identificados por anotações (E1 a E4, T1 a T18, L1 a L4), e as fases de extração e de carregamento estão encapsuladas em metanodes.

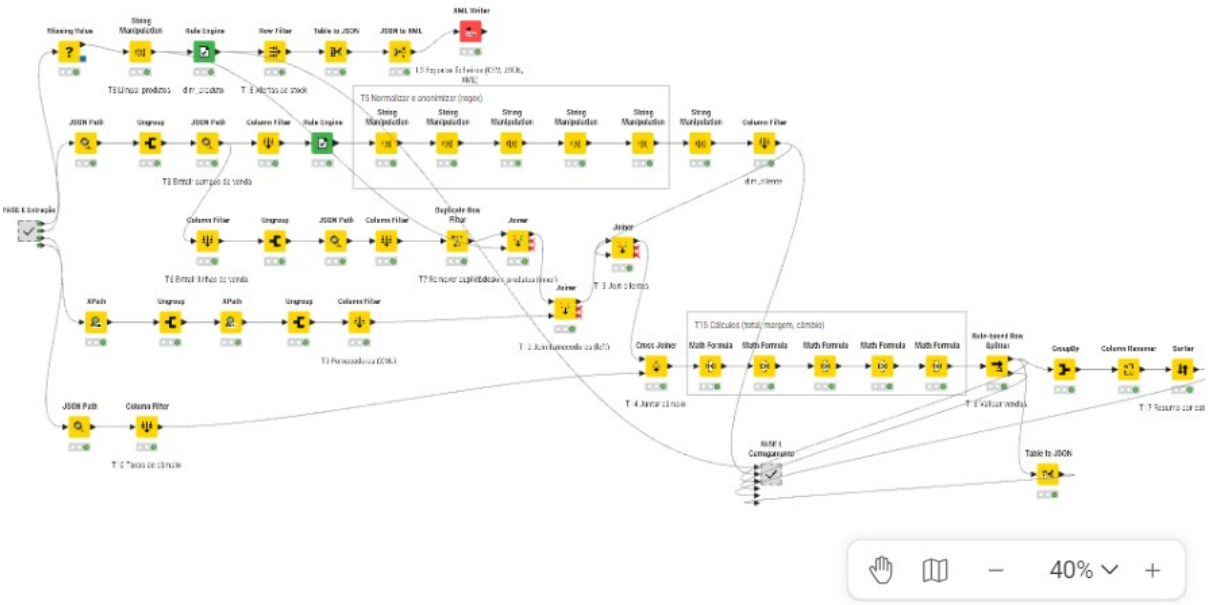


Figura 2: Workflow KNIME TP1_Ecommerce_ETL

Extração (metanode FASE E)

Bloco	Nós	Descrição
E1	CSV Reader	Lê `produtos_stock.csv` com delimitador `;` (8 linhas, 7 colunas)
E2	JSON Reader	Lê `vendas_online.json` para uma única célula JSON
E3	XML Reader	Lê `fornecedores_precos.xml` para uma célula XML
E4	GET Request	Chama `https://api.frankfurter.dev/v1/latest?`

Bloco	Nós	Descrição
		`base=EUR&symbols=USD,GBP`

Tabela 5: Nós de extração no KNIME



Figura 3: Metanode FASE E (extração)

Transformação

Desdobramento das vendas (T1 a T3, T6, T7). O JSON é processado em dois níveis: um JSON Path com `$[*]` seguido de Ungroup gera uma linha por transação; um segundo JSON Path extrai os campos do cliente e a lista `linha_venda[*]`. Num ramo separado, um novo Ungroup gera uma linha por produto vendido (5 linhas), um JSON Path extrai `sku`, `quantidade` e `preco_aplicado`, e o Duplicate Row Filter garante que não há linhas repetidas para o mesmo par transação e produto.

Fornecedores (T9). O XPath `/fornecedores/fornecedor` devolve uma coleção de nós, desagrupada para uma linha por fornecedor. Um segundo XPath extrai o identificador e o nome (atributos do fornecedor) e as listas de SKU, preço de custo e prazo. Um Ungroup aplicado às três listas em simultâneo produz 6 linhas, mantendo cada produto associado ao fornecedor correto.

Câmbio (T10). Um JSON Path extrai `$.date`, `$.rates.USD` e `$.rates.GBP` da resposta da API.

Limpeza dos produtos (T8). O nó Missing Value substitui o stock em falta por 0 e um String Manipulation corrige a categoria com `regexReplace($Categoria$, "Monitroes", "Monitores")`.

Validação, normalização e anonimização dos clientes (T4, T5). Um Rule Engine cria a coluna booleana `email_valido` e uma sequência de String Manipulation aplica as expressões regulares da Tabela 6. No

fim, um Column Filter remove todos os dados pessoais originais, ficando apenas as versões anonimizadas (tabela `dim_cliente`).

Objetivo	Expressão (KNIME)	Resultado
Validar email	`^[A-Za-z0-9._+-]+@[A-Za-z0-9-]+[.][A-Za-z.]+$`	TRUE / FALSE
Normalizar NIF	`regexReplace($contribuinte$, "^PT", "")`	234567890
Normalizar telefone	`regexReplace(regexReplace($telefone$, "[^0-9]", ""), "^(351)?([0-9]{9})$", "+351$2")`	+351912345678
Anonimizar NIF	`regexReplace($nif$, "^([0-9]{3})[0-9]{5}([0-9])$", "$1*****$2")`	234*****0
Anonimizar telefone	`regexReplace($telefone_norm$, "^([+]351[0-9]{3})[0-9]{3}([0-9]{3})$", "$1***$2")`	+351912***678
Anonimizar email	`regexReplace($email$, "^(.).*(@.*)$", "$1***$2")`	j***@email.com
Anonimizar nome	`regexReplace($nome$, "^([^ ]+) .*([A-Z])[^ ]*$", "$1 $2.")`	João S.

Tabela 6: Expressões regulares de validação, normalização e anonimização

Rows: 3 | Columns: 8 Sorting Filter Table Statistics Search

#	RowID	id_transacao (String)	data_venda (String)	metodo_paga... (String)	email_valido (Boolean)	nif_anon (String)	telefone_anon (String)	email_anon (String)	nome_anon (String)
1	Row0...	TXN-2026-001	2026-10-01 10:15...	MBWAY	true	234*****0	+351912***678	j***@email.com	João S.
2	Row0...	TXN-2026-002	2026-10-01 11:42...	VISA	false	210*****4	+351961***567	maria_santos_em...	Maria S.
3	Row0...	TXN-2026-003	2026-10-02 09:05...	MULTIBANCO	true	509*****3	+351933***777	c***@empresa.pt	Carlos O.

Figura 4: Dados de clientes depois da anonimização

Junções (T11 a T14). As linhas de venda são enriquecidas com três Joiners e um Cross Joiner:

- **inner join** com os produtos limpos (pelo SKU), para obter o nome, a categoria e o armazém;
- **left outer join** com os fornecedores, para não perder o SKU-1006, que não tem fornecedor;
- **inner join** com os clientes (pelo identificador da transação), para obter `email_valido`;
- **cross join** com a tabela de câmbio (uma linha), para que cada venda receba as taxas do dia.

Cálculos (T15). Cinco nós Math Formula calculam `total_linha`, `custo_total`, `margem`, `total_linha_usd` e `total_linha_gbp`.

Validação das vendas (T16). O Rule-based Row Splitter aplica as regras `MISSING $preco_custo$ => FALSE` e `$email_valido$ = TRUE => TRUE`. A primeira saída contém as 2 vendas válidas (`fact_vendas`) e a segunda as 3 rejeitadas.

Rows: 2 | Columns: 21 Sorting Filter Table Statistics Search

#	RowID	id_transacao (String)	data_venda (String)	metodo_pa... (String)	sku (String)	quantidade (Number (Inte...)	preco_aplic... (Number (Float))	Nome_Prod... (String)	Categoria (String)	Armazem_ID (String)	fornecec (String)
1	Row0...	TXN-2026-001	2026-10-01 1...	MBWAY	SKU-1001	2	79.9	Teclado Meca...	Perifericos	ARM-NORTH	TechSupp
2	Row0...	TXN-2026-003	2026-10-02 0...	MULTIBANCO	SKU-1004	2	189.5	Cadeira Gami...	Mobiliario	ARM-SOUTH	Global Offi

Figura 5: Vendas válidas (primeira saída do Rule-based Row Splitter)

Resumo por categoria (T17). GroupBy por categoria com soma de unidades, vendas e margem e contagem de vendas, seguido de Column Renamer (mapeamento de schema) e Sorter por total de vendas, por ordem decrescente.

Alertas de stock (T18). Um Rule Engine classifica cada produto (`dim_produto`) e um Row Filter mantém apenas os produtos com estado diferente de OK.

Carregamento (metanode FASE L)

Bloco	Destino	Conteúdo
L1, L2	SQLite `ecommerce_knime.db`	5 tabelas: `dim_produto` (8), `dim_cliente` (3), `fact_vendas` (2), `vendas_rejeitadas` (3), `resumo_categoria` (2), gravadas com DB Writer e a opção de substituir a tabela existente
L3	`resumo_categoria.csv`	CSV Writer com separador `;`
L3	`vendas_rejeitadas.csv`	CSV Writer com separador `;`
L3	`fact_vendas_0.json`	Table to JSON + JSON Writer
L3	`alertas_stock_0.xml`	Table to JSON + JSON to XML + XML Writer (conversão de formato JSON → XML)
L4	`log_execucao.csv`	Uma linha por execução: nº de vendas válidas, total, nº de rejeitadas e data/hora (modo append)

Tabela 7: Destinos da fase de carregamento no KNIME

O sufixo `_0` nos ficheiros JSON e XML é acrescentado automaticamente pelo KNIME, que permite gerar um ficheiro por linha da tabela de entrada.

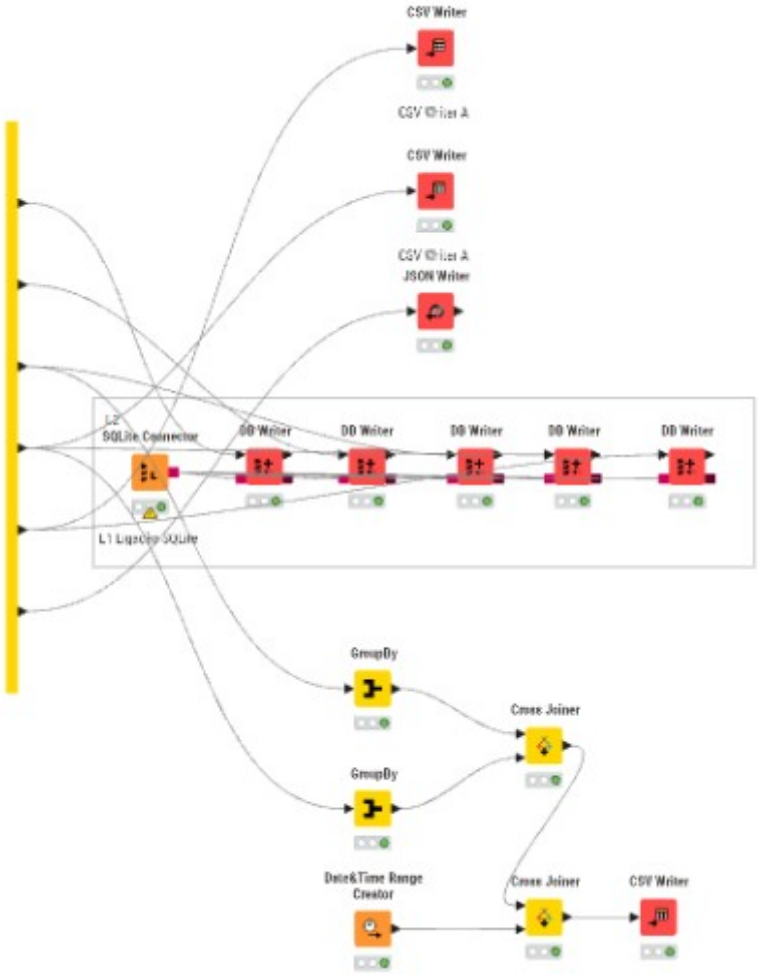


Figura 6: Metanode FASE L (carregamento)

As consultas de verificação da base de dados estão em `sql/knime_sqlite_consultas.sql`.

4.3 Resultados obtidos

As duas implementações produzem os mesmos resultados, o que valida a lógica de ambas:

Indicador	Pentaho	KNIME
Vendas válidas / rejeitadas	2 / 3	2 / 3
Total de vendas válidas	538,80 €	538,80 €
Mobiliario (vendas / margem)	379,00 € / 139,00 €	379,00 € / 139,00 €
Perifericos (vendas / margem)	159,80 € / 55,80 €	159,80 € / 55,80 €
Alertas de stock	4 (2 RUTURA, 2 CRITICO)	4 (2 RUTURA, 2 CRITICO)
Tabelas na base de dados	4	5 (inclui `dim_cliente`)

Tabela 8: Comparação dos resultados

As vendas rejeitadas são as duas linhas da transação TXN-2026-002 (email inválido) e a linha do SKU-1006 (produto sem fornecedor). Os valores em USD e GBP dependem do dia da execução, porque as taxas são obtidas em tempo real através da API.

O processo revelou ainda uma inconsistência entre sistemas: o SKU-1005 foi vendido apesar de ter stock 0, o que mostra o valor de um processo ETL na deteção de problemas de integração entre o sistema de vendas e o de armazém.

5. Jobs

5.1 Pentaho: job_etl.kjb

No Pentaho, a orquestração é feita pelo job `pentaho/job_etl.kjb`, que executa a transformação e reage ao resultado:

1. **Start** inicia o job.
2. **Executar ETL** corre a transformação `01_ler_dados.ktr`.
3. Em caso de sucesso, **Email Sucesso** envia uma notificação por email (SMTP do Gmail, porta 587) e o job termina em **Success**.
4. Em caso de falha, **Registar Erro** escreve no log, **Email Erro** envia um alerta por email e **Abort job** termina o job com erro.

O registo detalhado de cada execução fica em `output/pentaho/job_etl.log`. Por segurança, as credenciais de email não estão incluídas no repositório.

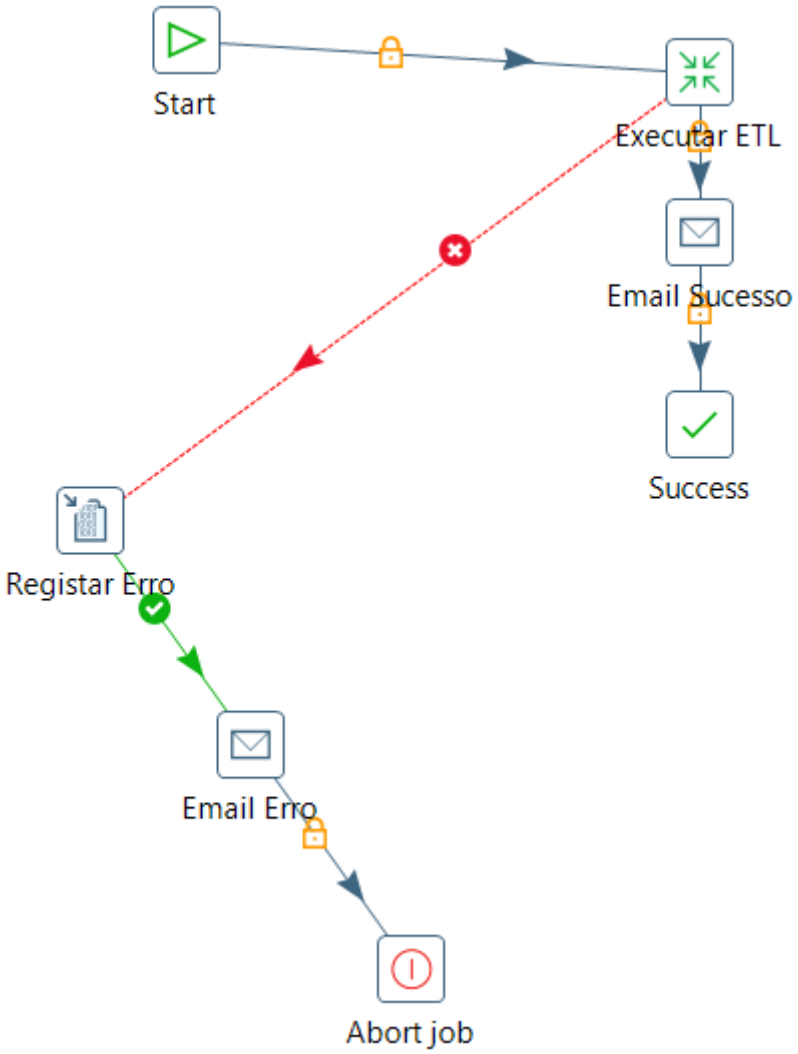


Figura 7: Job Pentaho job_etl.kjb

5.2 KNIME: orquestração com metanodes

O KNIME não tem o conceito de job separado das transformações. A orquestração é feita pelo próprio workflow:

- **metanodes por fase:** a extração (FASE E) e o carregamento (FASE L) estão encapsulados em metanodes, e a transformação está organizada em blocos anotados;
- **ordem de execução:** é definida pelas ligações entre nós; cada nó só executa quando os nós de que depende terminaram, e "Execute all" corre o processo completo;
- **registo:** o bloco L4 grava uma linha no `log_execucao.csv` em cada execução, com as contagens e a data e hora.

6. Vídeo de demonstração

O vídeo mostra a execução dos dois processos (job do Pentaho e workflow do KNIME) e os ficheiros e a base de dados gerados.



Figura 8: QR code de acesso ao vídeo

Link: https://youtu.be/VccxcmJV_bY

7. Conclusão e trabalhos futuros

O trabalho permitiu implementar um processo ETL completo, desde a recolha de dados em quatro fontes heterogéneas (CSV, JSON, XML e uma API REST) até à disponibilização dos resultados numa base de dados SQLite e em ficheiros CSV, JSON e XML. Pelo caminho foram aplicadas as principais operações de qualidade de dados pedidas no enunciado: tratamento de valores em falta, correção de valores, remoção de duplicados, validação e normalização com expressões regulares, anonimização de dados pessoais, junções, cálculos, filtragem e agregação.

Implementar o mesmo processo em duas plataformas foi particularmente útil. Os resultados idênticos (538,80 € em vendas válidas, as mesmas 3 rejeições e os mesmos 4 alertas de stock) validam a lógica de cada implementação. Ao mesmo tempo, ficaram claras as diferenças entre as ferramentas:

- o **Pentaho** separa claramente transformações e jobs, o que facilita a orquestração, o tratamento de erros e as notificações por email;
- o **KNIME** permite ver o resultado de cada nó imediatamente, o que torna o desenvolvimento e a depuração mais rápidos, e lida bem com dados aninhados (coleções JSON e XML) através do nó Ungroup.

Como trabalho futuro, o grupo identifica as seguintes melhorias:

- notificação por email e tratamento de erros (Try/Catch) também no KNIME;
- dashboard em tempo real (por exemplo, Grafana ou Node-RED Dashboard) alimentado pela base de dados;
- agendamento automático do processo e recolha contínua de vendas através de MQTT;
- migração da base de dados para PostgreSQL e caminhos relativos no workflow KNIME.

8. Referências

[1] Ribeiro, Ó. (2026). Integração de Sistemas de Informação, Trabalho Prático I: enunciado. IPCA.

[2] KNIME AG. KNIME Analytics Platform Documentation. https://docs.knime.com/

[3] KNIME AG. KNIME Hub: documentação dos nós. https://hub.knime.com/

[4] Hitachi Vantara. Pentaho Data Integration Documentation. https://docs.hitachivantara.com/

[5] Frankfurter. Frankfurter API (v1). https://frankfurter.dev/v1

[6] SQLite Consortium. SQLite Documentation. https://www.sqlite.org/docs.html

[7] União Europeia (2016). Regulamento (UE) 2016/679, Regulamento Geral sobre a Proteção de Dados.

[8] Repositório do projeto: https://github.com/braguex10/ISI-TP1-Ecommerce-ETL